

SDE-3 Backend Take-Home — Bulk Stage Move

Why we give this

Opportunities is the deal and pipeline engine at the centre of our CRM. One of the most demanding operations in it is the one customers reach for constantly: select a large set of deals and move them all at once.

It sounds like a loop. It is not. At our volumes it is a background job that must be idempotent, resumable, and polite to everything running alongside it — and it has to stay correct while individual users are editing the same records by hand.

That single operation is the whole exercise. Everything else in this brief exists only to give it somewhere to run. We would rather see one hard thing done with real care than a broad CRM slice done thinly.

The domain

- A **workspace** is a tenant. All data is scoped to one workspace and never crosses.
 - A workspace has a **pipeline** with an ordered list of **stages** (e.g. *New Lead* → *Contacted* → *Qualified* → *Proposal Sent* → *Negotiation* → *Closed Won*).
 - An **opportunity** is a deal. It sits in exactly one stage at a time and carries at least: a name, a monetary value, a status (*open* / *won* / *lost* / *abandoned*), an owner, and created/updated timestamps.
 - Every stage change is recorded as a transition.
-

Part 1 — Foundation (keep this small)

Just enough surface for Part 2 to be real. **Do not polish this. We are not grading it beyond "it works."**

- Create an opportunity.
- Move a single opportunity to another stage, recording the transition.
- List opportunities in a stage, with simple pagination.

Tenant scoping via an *X-Workspace-Id* header is fine. No auth, no UI.

Part 2 — Bulk stage move (this is the exercise)

A user selects a filter — stage, owner, status, value range, date range — and moves **up to 50,000 matching opportunities** to a different stage in one action.

The API accepts the request and returns immediately with a job handle. The work happens in the background. The user can poll for progress.

Build it so that all five of these hold:

1. Idempotent

The same submitted request — retried by a flaky client, or replayed by your own infrastructure — must not double-apply. Show us where the dedupe state lives and why it survives a process restart.

2. Resumable

If the worker is killed at 30,000 of 50,000, the job finishes correctly when work resumes. Define what "correctly" means here, and what a half-applied job looks like to the user while it is in that state.

3. Observably progressing

A progress endpoint that reports real committed state, not an in-memory counter that lies after a restart. The user should be able to tell the difference between *slow*, *stuck*, and *failed*.

4. Correct under concurrent edits

While the job runs, users are moving individual opportunities by hand — including ones inside the job's filter set. Decide what happens in that collision, implement it, and defend it. Also: your filter matched a set at submission time; records enter and leave that set while you work. State whether you operate on the snapshot or the live set, and why.

5. Well-behaved

It must not starve interactive requests in the same workspace, and it must not affect other workspaces at all. Show the mechanism — a concurrency cap, chunk pacing, queue partitioning, a token bucket, whatever you chose. An implemented mechanism, not a sentence claiming one.

Data & measurement

Ship a seeding script producing:

- **1 large workspace** — 500,000 opportunities across 12 stages, unevenly distributed, spread over ~18 months
- **5 small workspaces** — a few thousand each, so isolation is testable

Then measure your own system and report:

	What to report
Bulk move of 50,000	total wall-clock and sustained throughput
Interactive requests during the bulk run	p95 / p99 in the same workspace, and in a different one
Kill and resume	time to completion, and proof the result is correct

State the hardware. Rough numbers honestly measured beat impressive numbers we cannot reproduce — and we will try to reproduce them.

Constraints

- **Language:** Node.js (TypeScript, NestJS preferred) or Go.
- **Datastore:** MongoDB or PostgreSQL as the primary store. Add Redis, Kafka or similar if your design needs it — justify each.
- **Runnable:** one documented command (`docker compose up` or equivalent) brings it up, seeds a small dataset, and runs the tests on a clean machine. If we cannot run it, we cannot grade it.
- **Tests:** not a coverage number. Tests on the parts that are genuinely hard — idempotency, resume, the concurrent-edit collision. One test that would fail if you removed your safety mechanism is worth more than fifty CRUD assertions.

Explicitly out of scope — do not build these

Any UI · real authentication · a board or pipeline summary view · per-stage counts or value rollups · rich filtering and sorting beyond what the bulk filter needs · downstream automation events and consumers · deployment, CI, Kubernetes · custom fields, contacts, notes, tasks · exhaustive validation or a perfect error taxonomy.

If you find yourself building any of the above, stop — you are spending the budget in the wrong place.

On using AI tools

Use them. We do. No penalty, no need to disclose which.

The only rule is that you own the output. In the review call we will pick parts of your code and ask why they are that way, what the alternative was, and what happens when the input is adversarial. Anything you cannot defend, we will treat as not yours — so read what you ship.

Deliverables

1. **A repository** — with real commit history, not one squashed commit.
 2. **README.md** — how to run it, how to seed, how to test, what is and is not implemented.
 3. **DESIGN.md** — 2–3 pages, and the most important thing you will submit:
 - How the job is chunked, and how the cursor survives a restart — plus the index that makes chunking cheap.
 - Your idempotency model: what the key is, where it is stored, what it protects, and where a retry could still slip through.
 - Your concurrency control on a single opportunity, and what happens when a manual move and the job hit the same record.
 - Snapshot vs live filter set, and the consequence of your choice.
 - Your isolation mechanism, and the hole it still leaves.
 - **What breaks at 10×** — a 500,000-record move, 20M opportunities in the workspace. Name the *first* thing to fail and what you would do about it.
 - **What you would do with another week**, ranked.
 4. **BENCHMARKS.md** — the numbers above, the hardware, the method.
-

How we evaluate

In rough order of weight: the bulk job's architecture and resumability; correctness under concurrency and retry; isolation under load; honest measurement; then code quality and clarity of the write-up.

A submission where the job is correct, restartable and well-behaved — and the write-up knows exactly where it is still weak — scores far higher than one with more endpoints.

Questions

If anything is ambiguous, ask us, or make a reasonable assumption and write it down. Documenting an assumption is never wrong. Silently guessing is.

Good luck — we are looking forward to reading it.